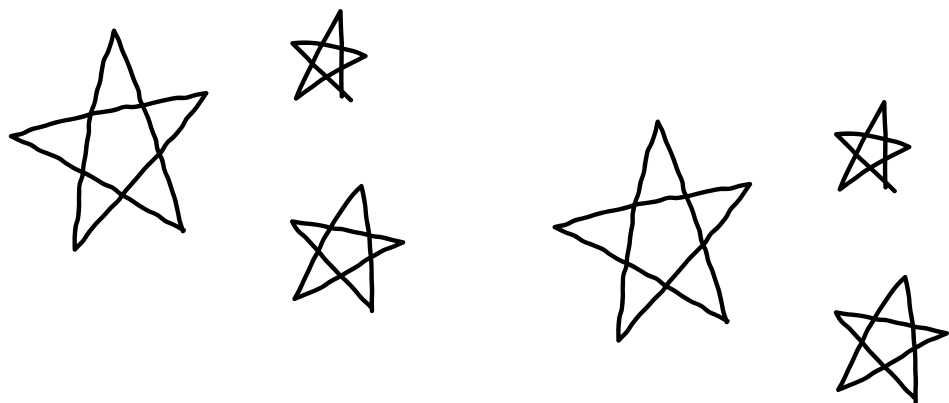
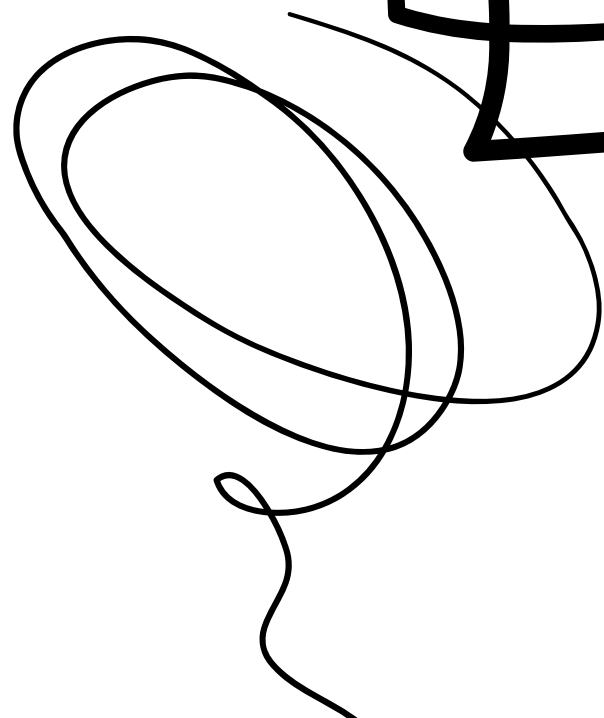
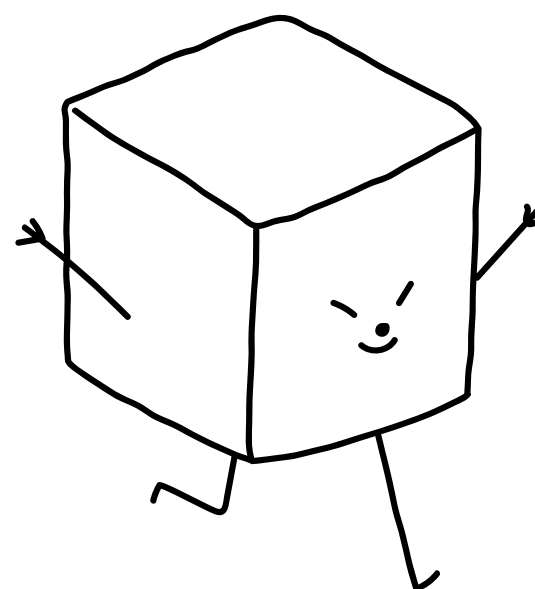
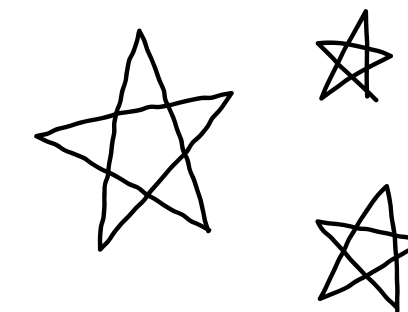
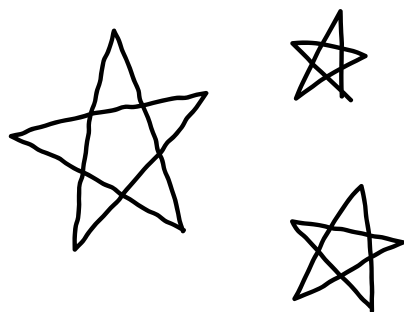


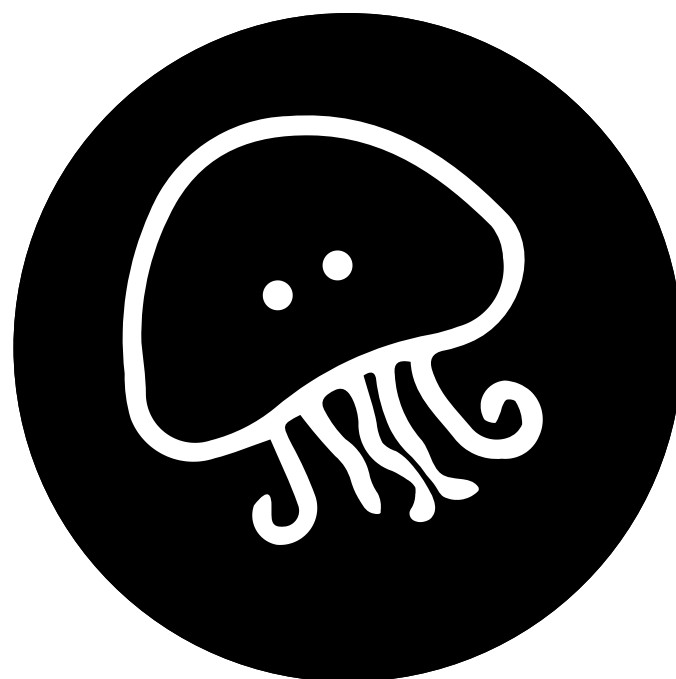
QLoRA Text-to-SQL

by HFT group

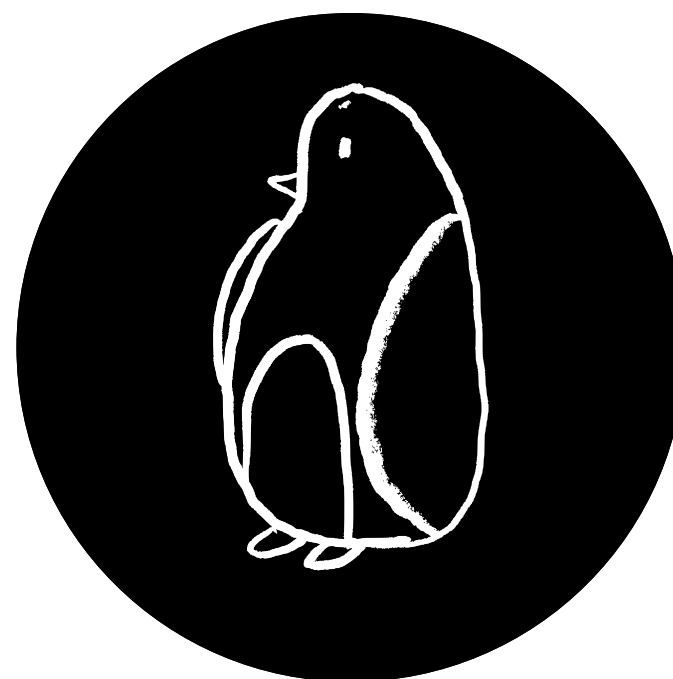




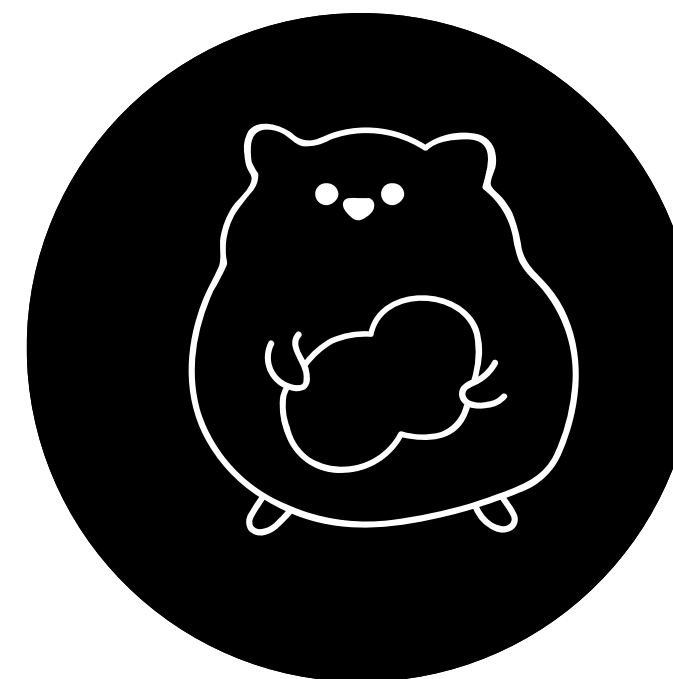
Наша команда



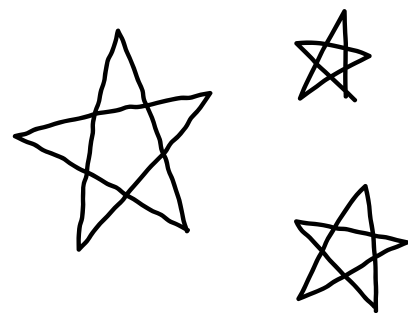
Плешаков
Василий



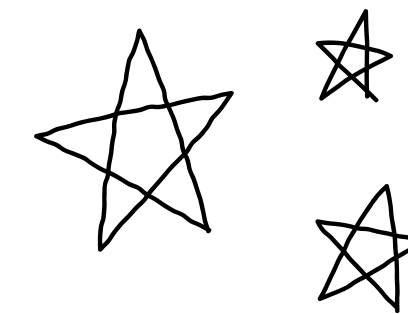
Маханькова
Ольга



Просвирина
Мария

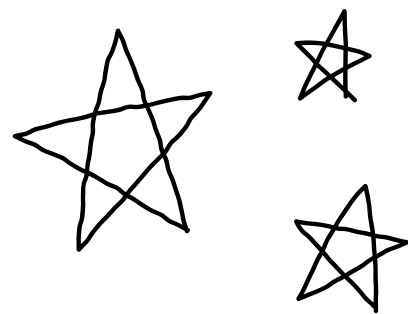


Цель проекта

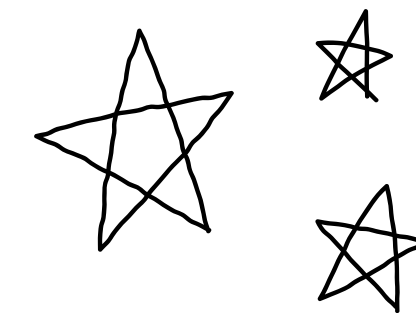


Цель проекта – исследовать, насколько эффективно можно адаптировать относительно небольшую языковую модель под Text-to-SQL

Text-to-SQL – это задача преобразования вопроса на естественном языке в SQL-запрос к базе данных. Модель получает вопрос и схему базы данных, определяет нужные таблицы, колонки, условия и связи между таблицами, после чего генерирует DQL-запрос, который можно выполнить в СУБД

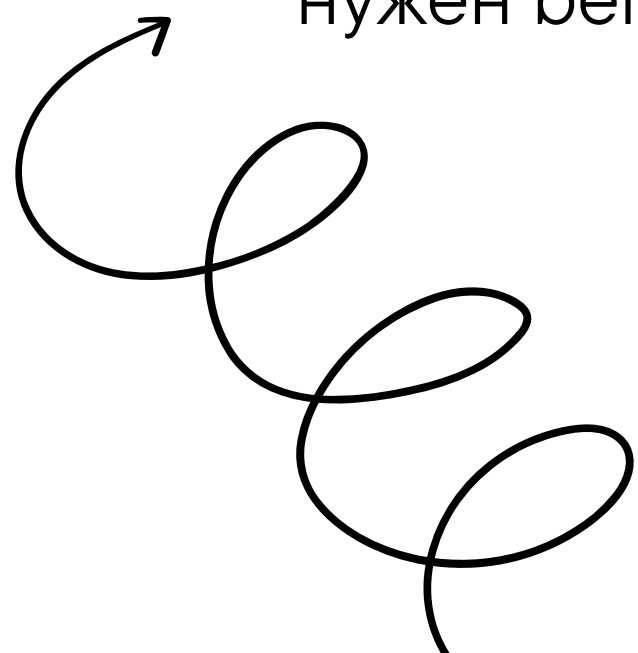


Проблематика



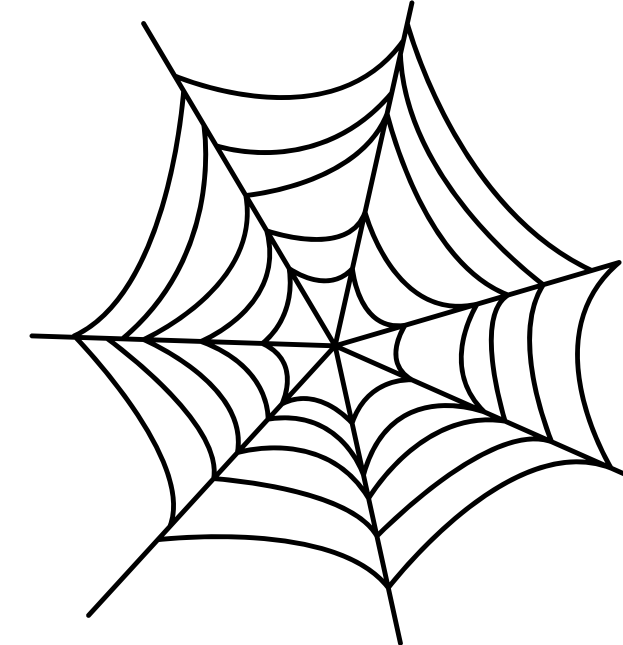
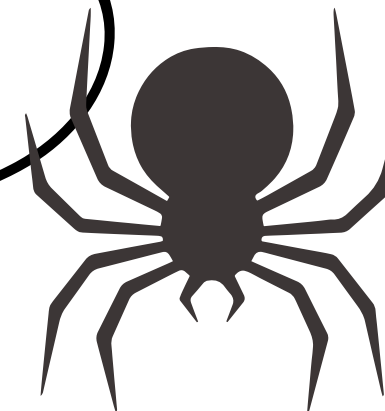
Задача **Text-to-SQL** требует не только синтаксически корректной генерации, но и понимания структуры конкретной базы данных. Модель должна уметь выбирать нужные таблицы и колонки, учитывать связи между сущностями, строить JOIN и корректно обрабатывать фильтрацию, группировку, вложенные запросы и set-операции

При этом важно оценивать не запоминание готовых шаблонов, а способность переносить знания на новые схемы и предметные области. Поэтому для обучения и контроля качества нужен benchmark с разнообразными базами данных, запросами разного уровня сложности и отдельной оценкой качества на незнакомых схемах





Данные для обучения



Для обучения используется датасет **Spider 1.0** – набор примеров для Text-to-SQL с базами данных из разных предметных областей и SQL-запросами разного уровня сложности. Spider содержит 10 181 вопрос, 5 693 уникальных SQL-запроса и 200 баз данных из 138 предметных областей

Для каждого примера по db_id извлекается схема базы данных. Затем она преобразуется в формат CREATE TABLE с указанием колонок, первичных и внешних ключей. В результате модель получает инструкцию, схему базы данных и вопрос, а целевым ответом является SQL-запрос.

После обработки данных мы получили 8 659 примеров для обучения и 1 034 примера для финальной оценки.

Instruction: You are a Text-to-SQL assistant. Generate a valid SQLite SQL query for the given question using only the provided database schema. Do not use tables or columns that are not present in the schema.

Return only the SQL query

Database schema:

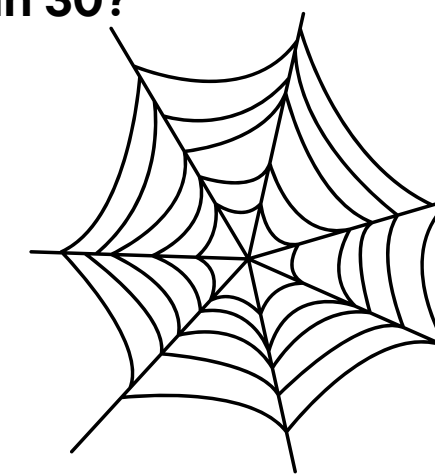
```
CREATE TABLE singer (  
  singer_id INTEGER PRIMARY KEY,  
  name TEXT,  
  country TEXT,  
  age INTEGER);
```

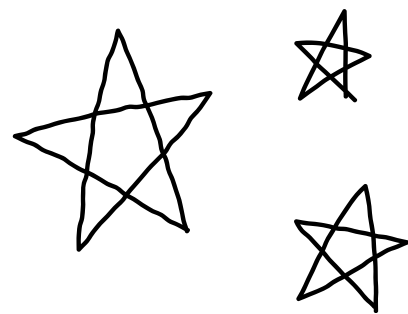
Question:

How many singers are older than 30?

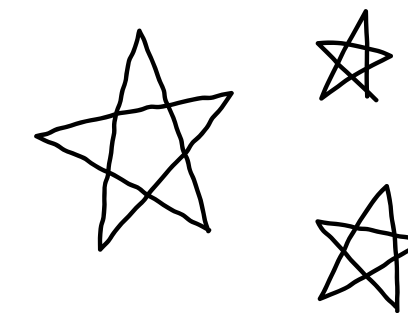
SQL:

```
SELECT count(*)  
FROM singer  
WHERE age > 30;
```





QLoRA-дообучение



В качестве базовой модели используется **Qwen2.5-Coder-1.5B-Instruct** – компактная языковая модель, ориентированная на работу с кодом.

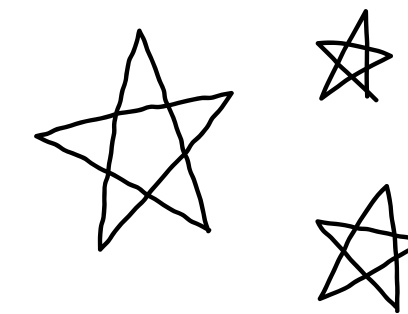
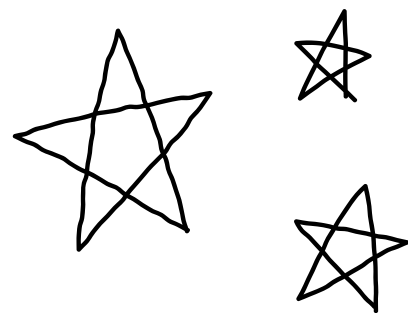
Для адаптации под задачу Text-to-SQL применяется **QLoRA**. Базовые веса модели загружаются в 4-битном виде и остаются замороженными, а в процессе обучения обновляются только небольшие LoRA-адаптеры, добавленные к отдельным слоям: attention-проекции: q_proj, k_proj, v_proj, o_proj; MLP-проекции: gate_proj, up_proj, down_proj.

Вместо изменения больших матриц весов модель обучает низкоранговые поправки:

$$\mathbf{W} = \mathbf{W}_0 + \mathbf{BA},$$

где $\mathbf{W}_0$ – замороженные веса базовой модели, а $\mathbf{A}$ и $\mathbf{B}$ – компактные обучаемые матрицы.

Такой подход позволяет значительно снизить требования к GPU-памяти и дообучать модель на ограниченных вычислительных ресурсах без полного fine-tuning



Для оценки модели используются официальные метрики Spider

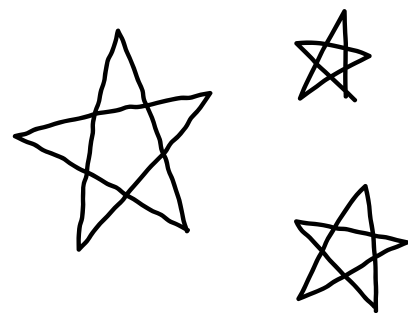
Execution Accuracy показывает, возвращает ли сгенерированный SQL тот же результат, что и эталонный запрос. Для каждого примера выполняются оба запроса к базе данных, после чего сравниваются полученные ответы.

Exact Match проверяет структурное совпадение с эталонным SQL. Запросы предварительно разбираются на компоненты, поэтому различия в регистре, форматировании и порядке некоторых элементов не считаются ошибкой. Метрика более строгая, чем Execution Accuracy: эквивалентные по результату, но по-разному построенные запросы могут не совпасть.

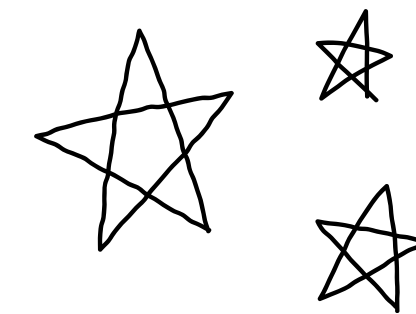
Partial Matching оценивает отдельные части SQL-запроса: SELECT, WHERE, GROUP BY, ORDER BY, ключевые слова и set-операции UNION, INTERSECT, EXCEPT. Для каждого компонента рассчитываются precision, recall и F1-score. Это позволяет понять, на каких конструкциях модель ошибается чаще всего.

Test Suite Accuracy проверяет семантическую корректность запроса на наборе автоматически сгенерированных тестовых баз данных. Запрос считается правильным, только если его результаты совпадают с эталоном во всех тестовых сценариях. Это снижает вероятность случайного совпадения результатов на одной базе данных.

Дополнительно качество анализируется отдельно по уровням сложности запросов: **easy, medium, hard и extra**.



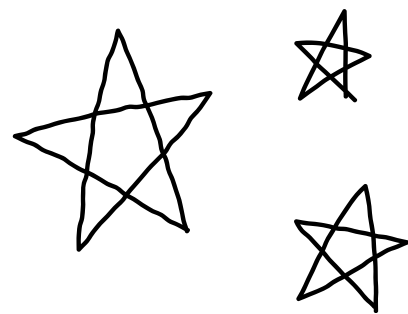
Оценка качества



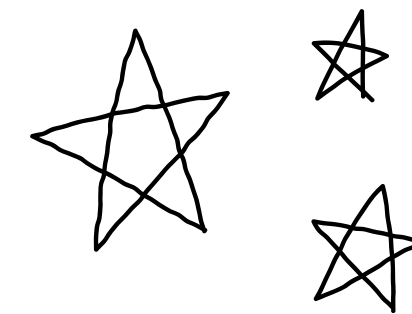
Стоит отметить, что качество модели оценивается не только по официальным метрикам Spider. После получения предсказаний проводится дополнительный анализ ошибок, допущенных при генерации SQL-запросов.

Качество решения задачи Text-to-SQL важно оценивать не только по итоговому числу, но и аналитически: необходимо понимать, какие типы ошибок встречаются чаще всего и на каких конструкциях модель теряет качество. В частности, отдельно рассматриваются лишние и отсутствующие таблицы и колонки в эталонном запросе; лишние или некорректные JOIN, ошибки в условиях фильтрации, агрегатах, сортировке, выборе и реализации set-операций.

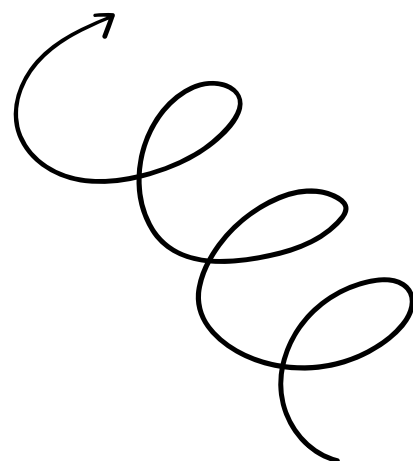
Такой анализ позволяет не просто зафиксировать результат модели, а определить направления для дальнейшего улучшения обучения и inference-пайплайна.



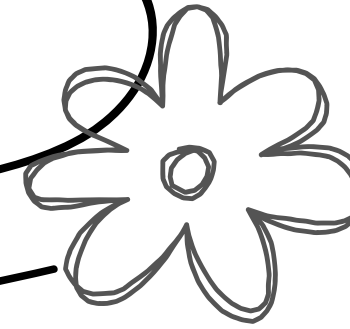
Pipeline



1. Загрузка данных
2. Загрузка модели в 4-битном виде
3. Претокенизация датасета для корректной валидации
4. Запуск обучения LoRa адаптеров
5. Выбор оптимальной модели
6. Инференс
7. Оценка



Загрузка модели в 4-битном виде

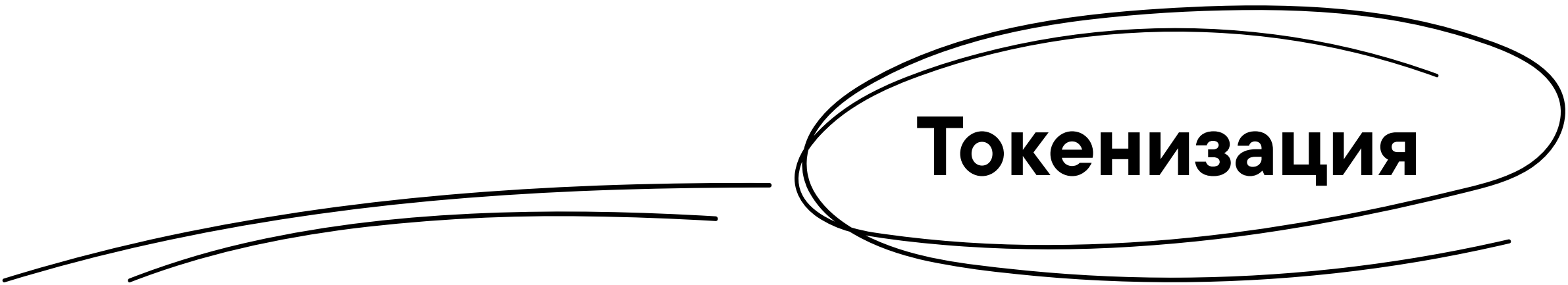


Базовая модель загружается с помощью библиотеки **Unsloth** через функцию **FastLanguageModel.from_pretrained()**

При загрузке включается 4-битная квантизация весов. Это означает, что параметры модели хранятся в более компактном формате, что значительно снижает расход GPU-памяти

Основные настройки загрузки:

- Выбор базовой модели – **Qwen2.5-Coder-1.5B-Instruct**
- Максимальная длина контекста – **2048**
- Автоматический выбор типа вычислений – float16, выбранный для совместимости с GPU NVIDIA T4
- Включение 4-битной квантизации



Токенизация

Перед обучением все примеры заранее преобразуются в токены с помощью токенизатора из библиотеки **Transformers**.

Для каждого примера отдельно обрабатываются prompt и эталонный SQL-запрос, после чего они объединяются в одну последовательность. Инструкция, схема базы данных и вопрос используются как контекст, а ошибка обучения рассчитывается только по SQL-ответу.

Претокенизация выполняется через функцию подготовки примеров и метод **Dataset.map()** из библиотеки datasets. После обработки дополнительно проверяется длина последовательностей, чтобы SQL-ответ не был потерян при обрезании.

Запуск обучения адаптеров и выбор модели

Используемые параметры LoRA:

r = 16

lora_alpha = 32

lora_dropout = 0.0

bias = none

num_train_epochs = 1

learning_rate = 1e-4

weight_decay=0.01

per_device_train_batch_size = 4

gradient_accumulation_steps = 4

effective batch size = 16

optimizer = paged_adamw_8bit

scheduler = cosine

Для обучения применяется SFTTrainer из библиотеки TRL

Каждые 50 шагов рассчитывается train loss и сохраняется checkpoint. Это позволяет отслеживать качество обучения и выбирать модель до момента переобучения. Выбирается модель по оптимальному соотношению train loss + количеству шагов

Инференс и оценка

Проводим инференс модели, загружаем LoRA адаптеры, сначала проверяем результаты, на первых 50 запросах визуально, затем через официальный скрипт Spider проводим расчет метрик. Далее анализируем и визуализируем ошибки модели с помощью специального нами написанного скрипта, делаем выводы о дальнейших улучшениях качества предсказаний.



Qwen no LoRA

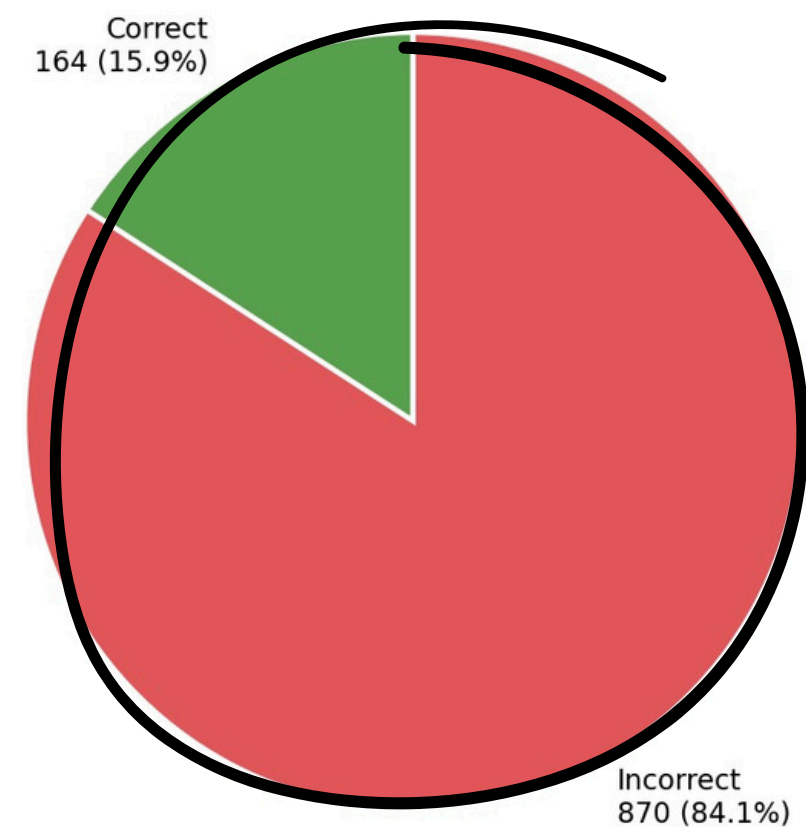
	easy 248	medium 446	hard 174	extra 166	all 1034
===== EXECUTION ACCURACY =====					
execution	0.363	0.327	0.230	0.211	0.301
===== EXACT MATCHING ACCURACY =====					
exact match	0.323	0.123	0.098	0.193	0.178
----- PARTIAL MATCHING ACCURACY -----					
select	0.863	0.706	0.730	0.739	0.760
select(no AGG)	0.863	0.739	0.743	0.739	0.775
where	0.361	0.250	0.290	0.143	0.281
where(no OP)	0.361	0.275	0.516	0.143	0.347
group(no Having)	0.700	0.609	0.605	0.741	0.654
group	0.600	0.517	0.558	0.707	0.587
order	0.739	0.434	0.611	0.759	0.620
and/or	1.000	0.902	0.929	0.881	0.927
IUEN	0.000	0.000	0.250	0.000	0.250
keywords	0.873	0.630	0.551	0.723	0.681

Сравнение baseline QLoRA
с Qwen

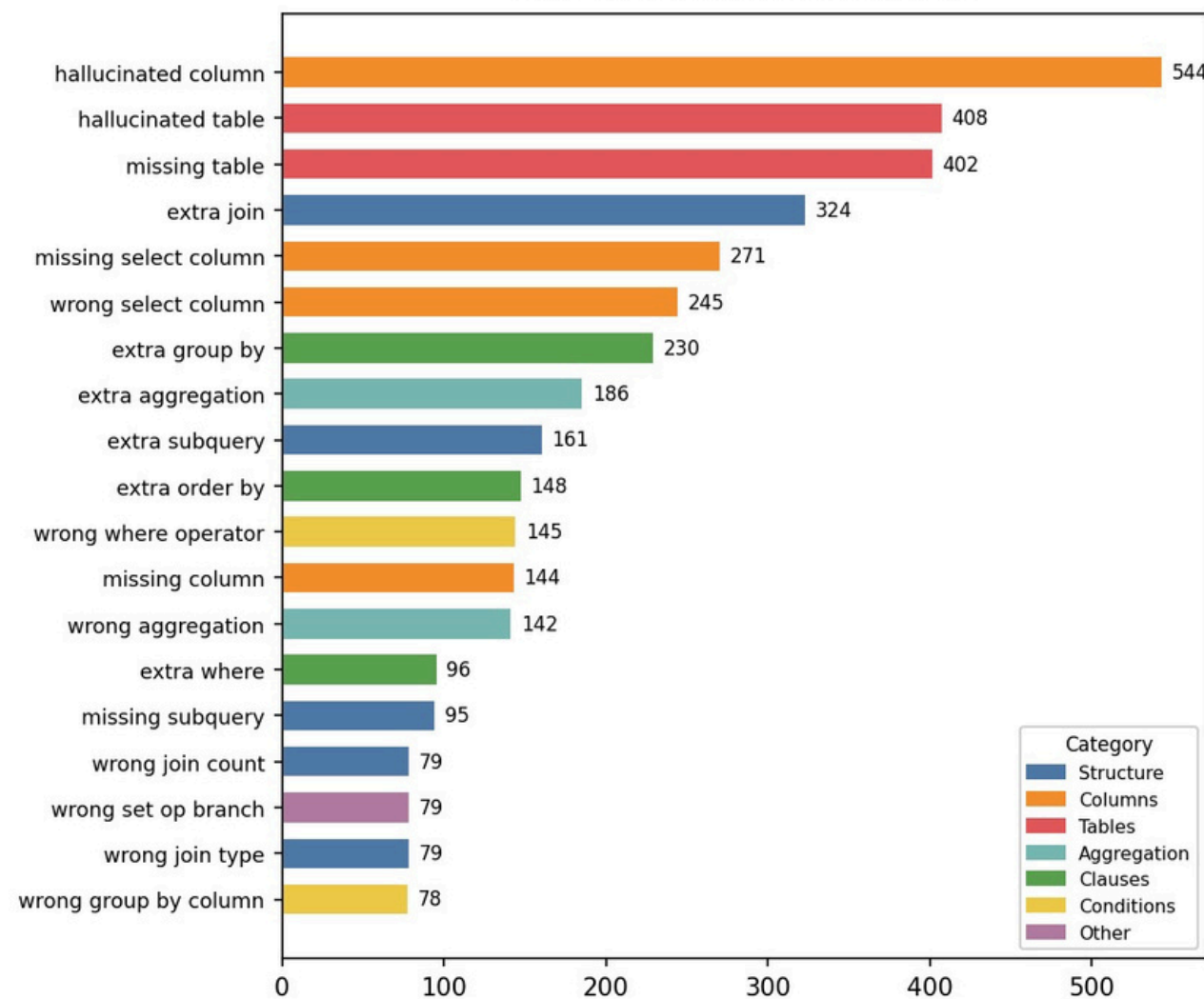
Qwen with base QLoRA

	easy 248	medium 446	hard 174	extra 166	all 1034
===== EXECUTION ACCURACY =====					
execution	0.883	0.735	0.500	0.440	0.684
===== EXACT MATCHING ACCURACY =====					
exact match	0.835	0.637	0.397	0.380	0.603
----- PARTIAL MATCHING ACCURACY -----					
select	0.934	0.913	0.972	0.860	0.920
select(no AGG)	0.947	0.925	0.972	0.868	0.930
where	0.889	0.751	0.532	0.614	0.724
where(no OP)	0.907	0.763	0.597	0.729	0.764
group(no Having)	0.895	0.843	0.810	0.828	0.838
group	0.895	0.783	0.786	0.812	0.800
order	0.955	0.867	0.911	0.826	0.872
and/or	1.000	0.984	0.983	0.920	0.978
IUEN	0.000	0.000	0.364	0.519	0.415
keywords	0.967	0.949	0.765	0.816	0.896

Overall Accuracy



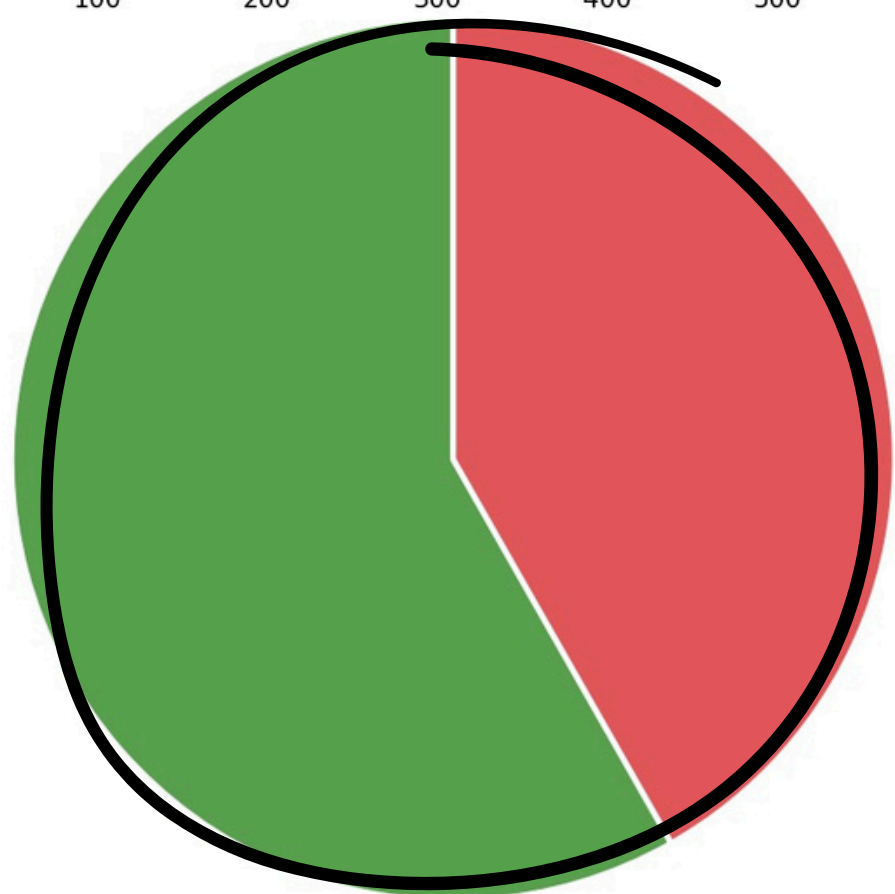
Error Type Distribution (Top 20)



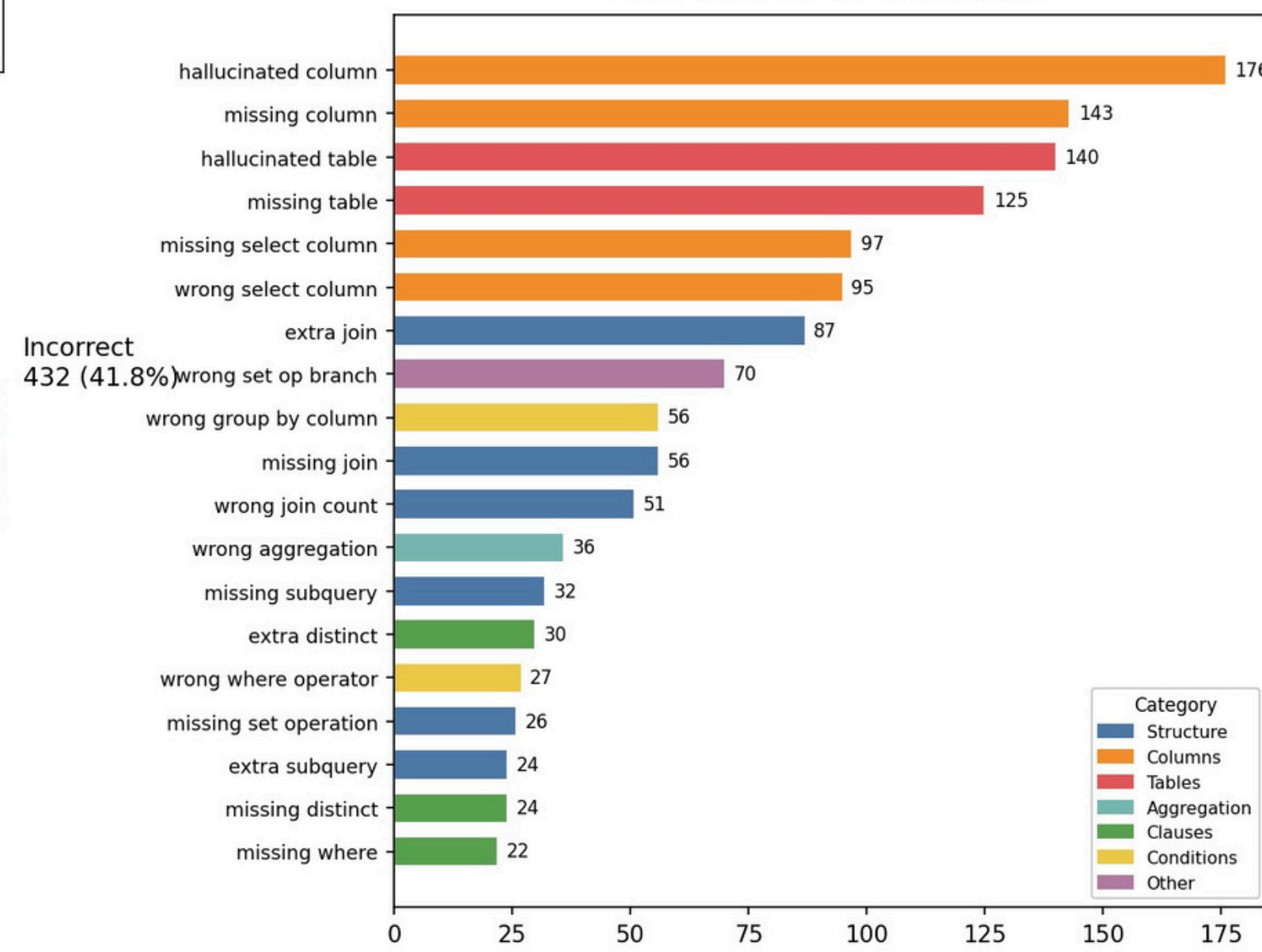
Qwen no LoRA

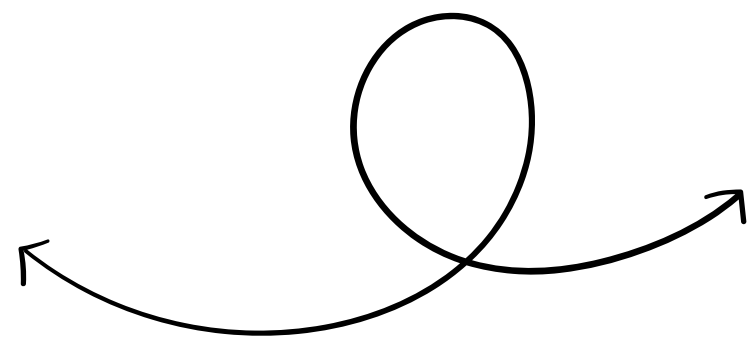
Qwen with base QLoRA

Correct
602 (58.2%)



Error Type Distribution (Top 20)





Попытки улучшения

На основе проведенного анализа, мы увидели, что модель чаще всего пропускает или добавляет лишние колонки, использует лишние JOIN, плохо справляется с запросами с set-операциями.

Также видна просадка качества предсказаний по официальным метрикам на запросах со сложностью выше medium.

Наши потуги заключались в основном в попытках добавить **новые улучшенные промты, дать большие веса запросам с большей сложностью и с их помощью сделать аугментацию датасета, добавив больше таких примеров-клонов.**

Сразу хочется отметить, что в результате аугментации были найдены артефакты типа:

CLEAN: SELECT avg(age) , min(age) , max(age) FROM singer WHERE country = "French"

GOLD: SELECT avg(age) , min(age) , max(age) FROM singer WHERE country = 'France',

хотя с ними справлялась даже модель без LoRA, а также была видна сильная просадка по приблизительным метрикам (первые 50 примеров), что убедило нас отказаться от данной идеи

Новые промты

Мы пробовали обучать модель на различных новых более сложных и конкретизированных промтах.

Первый содержал более полноценную задачу (упомянули лишние JOIN и другие ошибки) и уточненные связи между таблицами, второй был более короткий просто с улучшенными связями и ключами баз данных. По итогу оба промта оказались видимо слишком сложным для нашей небольшой модели, и по итогу мы просели по общим метрикам. Это произошло из-за того, что модель действительно начала немного лучше справляться с hard примерами, но ухудшила свое качество на легких и средних.

	easy	medium	hard	extra	all
count	248	446	174	166	1034
===== EXECUTION ACCURACY =====					
execution	0.847	0.673	0.506	0.416	0.645
===== EXACT MATCHING ACCURACY =====					
exact match	0.815	0.583	0.374	0.380	0.571
----- PARTIAL MATCHING ACCURACY -----					
select	0.942	0.898	0.956	0.907	0.921
select(no AGG)	0.951	0.914	0.956	0.915	0.931
where	0.898	0.719	0.525	0.585	0.707
where(no OP)	0.907	0.725	0.625	0.662	0.743
group(no Having)	0.810	0.855	0.838	0.889	0.856
group	0.762	0.800	0.838	0.852	0.815
order	0.900	0.877	0.900	0.862	0.880
and/or	1.000	0.982	0.971	0.900	0.972
IJEN	0.000	0.000	0.263	0.591	0.391
keywords	0.953	0.925	0.731	0.814	0.878



Когда мы поняли, что в обучении уже ничего улучшить нельзя и все наши идеи только ухудшают общий результат, то единственным вариантом, как улучшить предсказания небольшой модели осталась только работа с инференсом. Тогда был реализован дополнительный модуль SchemaGuard.

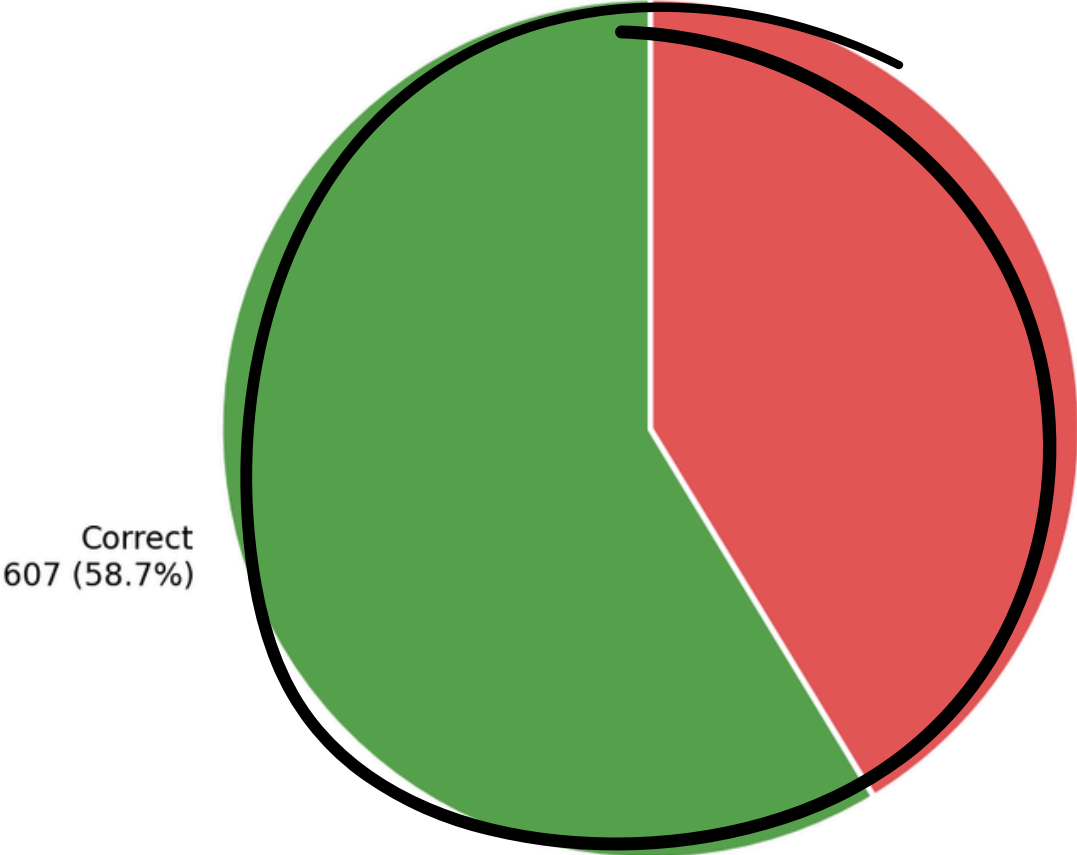
SchemaGuard используется как дополнительный этап после генерации SQL-запросов. Он проверяет, существуют ли указанные таблицы и колонки, корректно ли используются aliases и JOIN, а также может ли запрос быть выполнен в SQLite.

Если в запросе обнаруживается формальная ошибка, модель получает исходный SQL и краткий отчёт о найденных проблемах, после чего выполняет повторную генерацию. В наших экспериментах такой repair-pass действительно срабатывал: модель регенерировала часть запросов и в отдельных случаях исправляла структуру SQL. При этом изменения принимаются только после повторной проверки SchemaGuard, чтобы не заменить исходный запрос на менее корректный вариант.

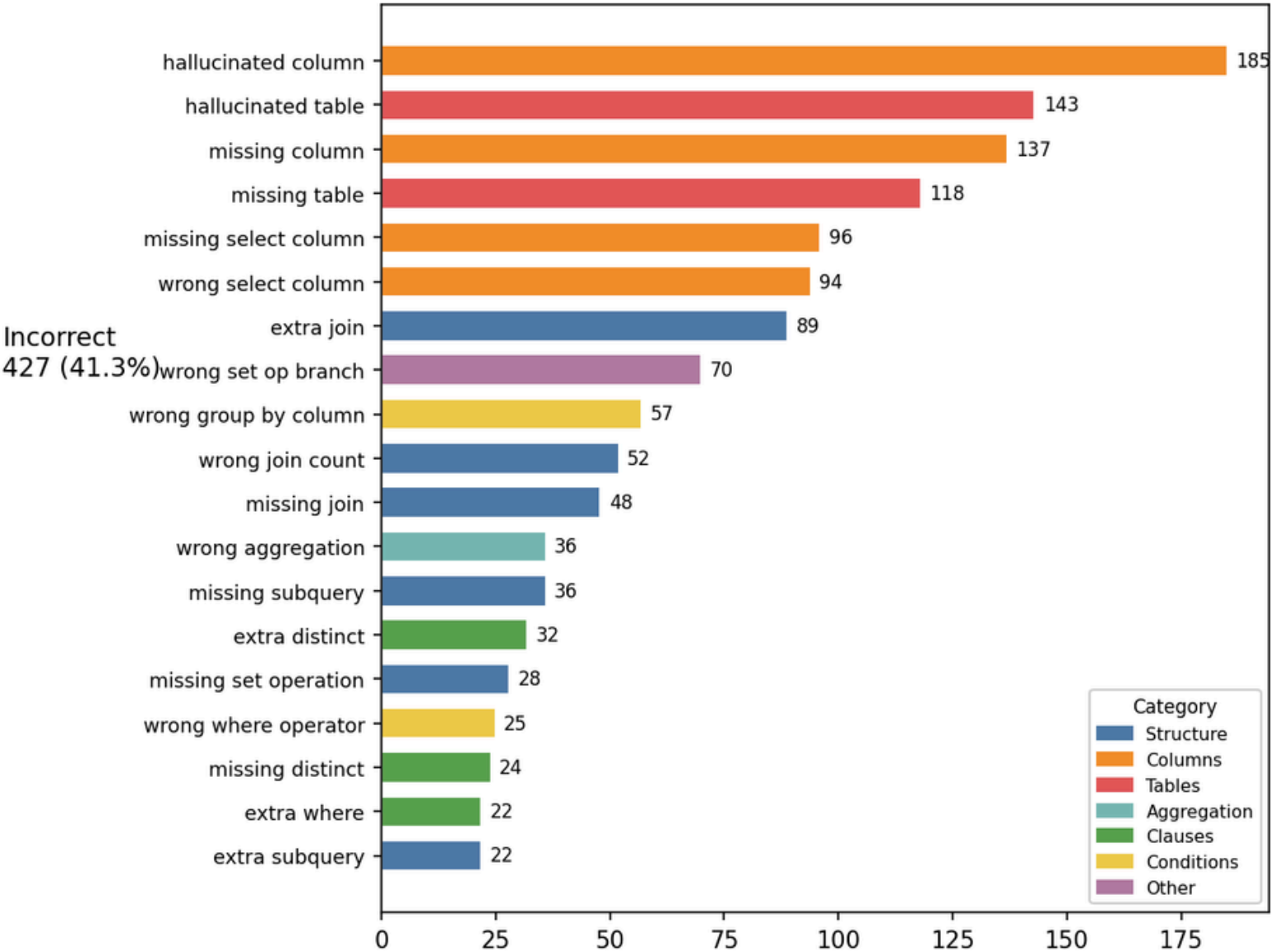
Новый улучшенный
инференс baseline

	easy	medium	hard	extra	all
count	248	446	174	166	1034
===== EXECUTION ACCURACY =====					
execution	0.883	0.747	0.506	0.446	0.691
===== EXACT MATCHING ACCURACY =====					
exact match	0.835	0.646	0.362	0.380	0.601
----- PARTIAL MATCHING ACCURACY -----					
select	0.931	0.908	0.973	0.861	0.917
select(no AGG)	0.943	0.920	0.973	0.869	0.927
where	0.889	0.742	0.473	0.609	0.711
where(no OP)	0.907	0.753	0.541	0.725	0.751
group(no Having)	0.895	0.853	0.791	0.821	0.837
group	0.895	0.793	0.767	0.806	0.800
order	0.955	0.875	0.891	0.817	0.867
and/or	1.000	0.984	0.988	0.920	0.979
IUEN	0.000	0.000	0.286	0.519	0.373
keywords	0.967	0.948	0.714	0.818	0.886

Overall Accuracy



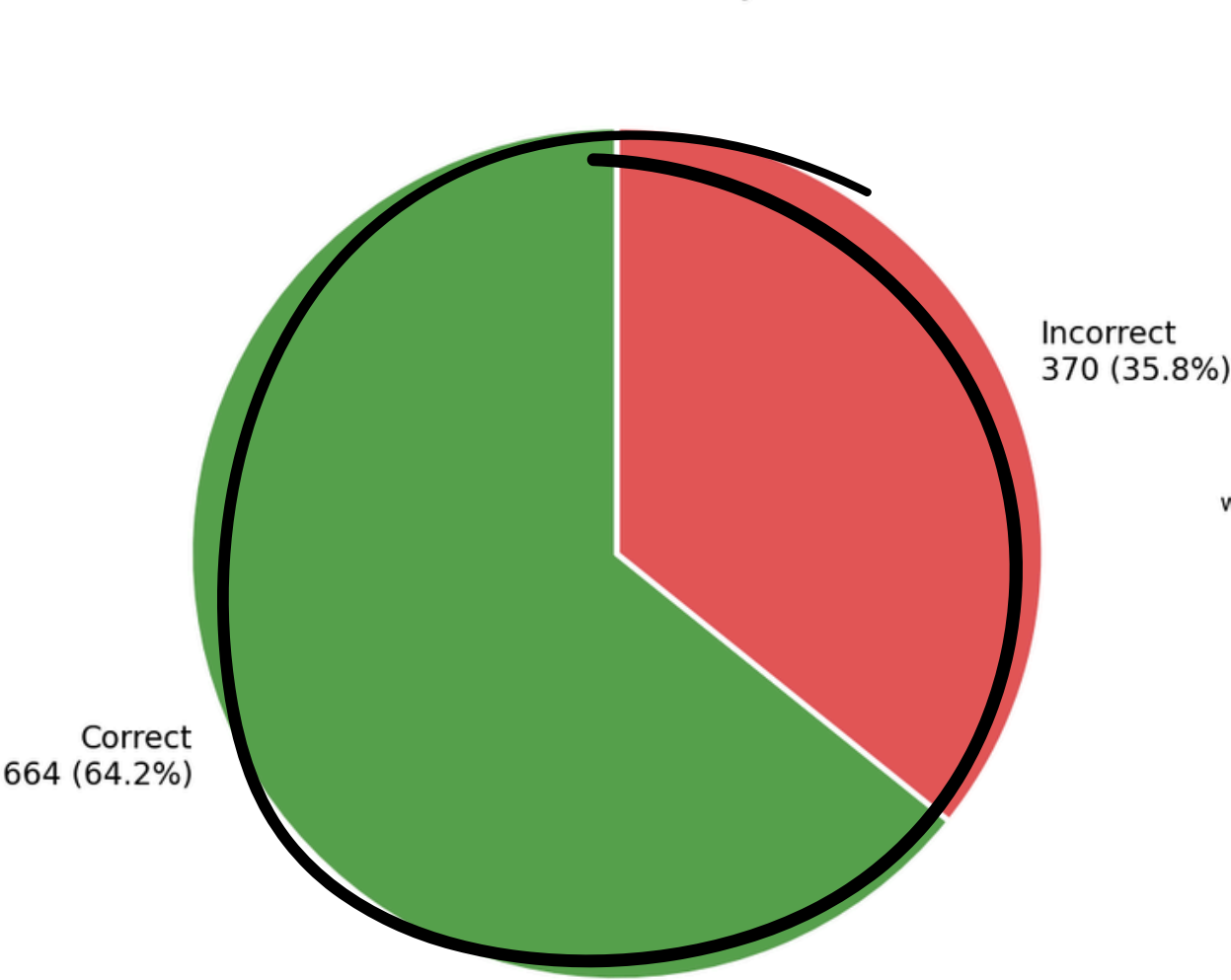
Error Type Distribution (Top 20)



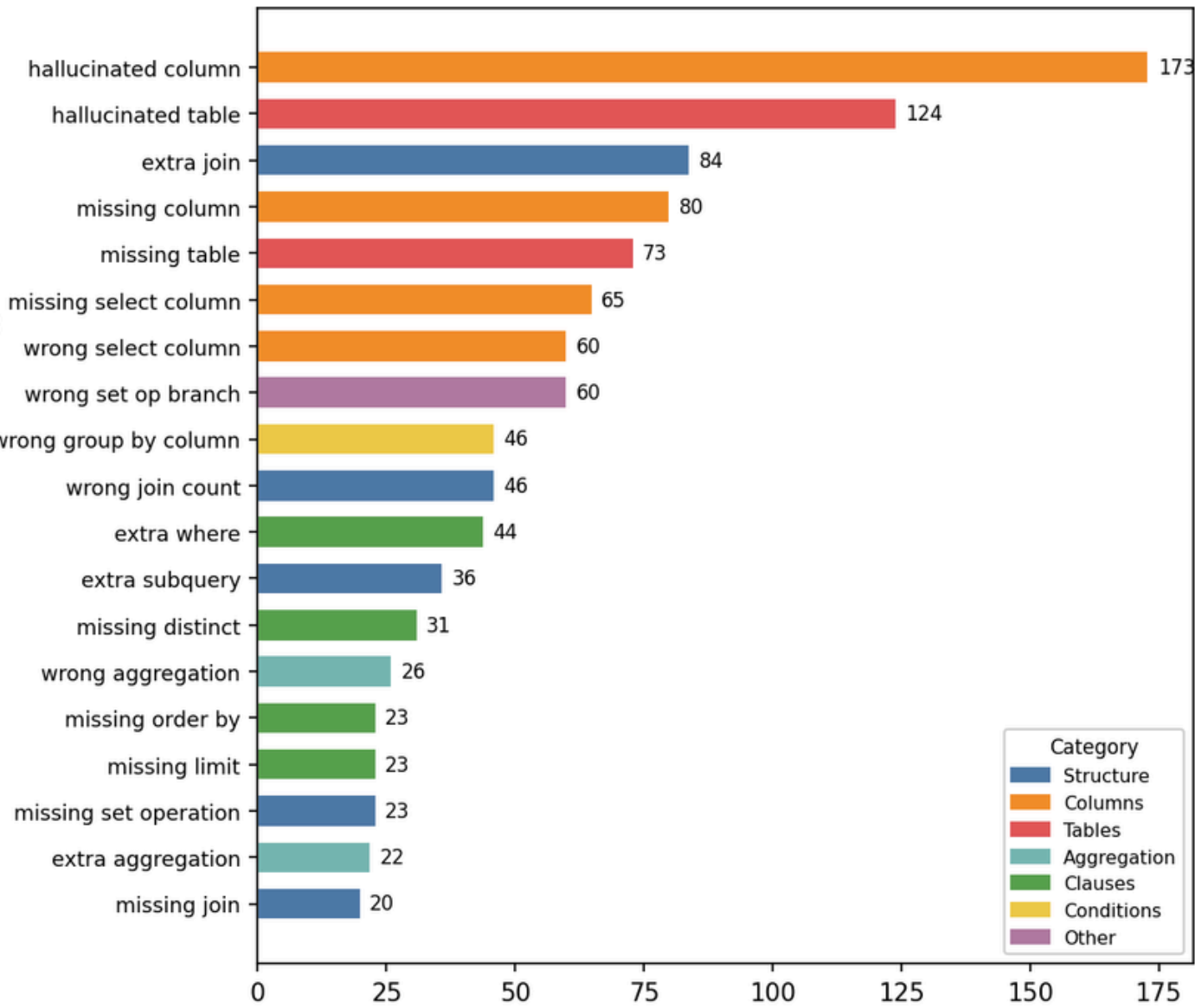
Qwen2.5-Coder-7B-Instruct

count	easy 248	medium 446	hard 174	extra 166	all 1034
===== EXECUTION ACCURACY =====					
execution	0.883	0.812	0.713	0.608	0.779
===== EXACT MATCHING ACCURACY =====					
exact match	0.847	0.744	0.609	0.452	0.699
----- PARTIAL MATCHING ACCURACY -----					
select	0.980	0.933	0.958	0.919	0.947
select(no AGG)	0.988	0.938	0.958	0.926	0.952
where	0.821	0.834	0.718	0.576	0.763
where(no OP)	0.830	0.845	0.757	0.729	0.803
group(no Having)	0.941	0.867	0.857	0.841	0.863
group	0.941	0.817	0.833	0.826	0.831
order	0.952	0.981	0.938	0.889	0.933
and/or	1.000	0.998	0.977	0.932	0.984
IUEN	0.000	0.000	0.500	0.643	0.545
keywords	0.953	0.944	0.840	0.846	0.907
----- PARTIAL MATCHING RECALL -----					

Overall Accuracy



Error Type Distribution (Top 20)



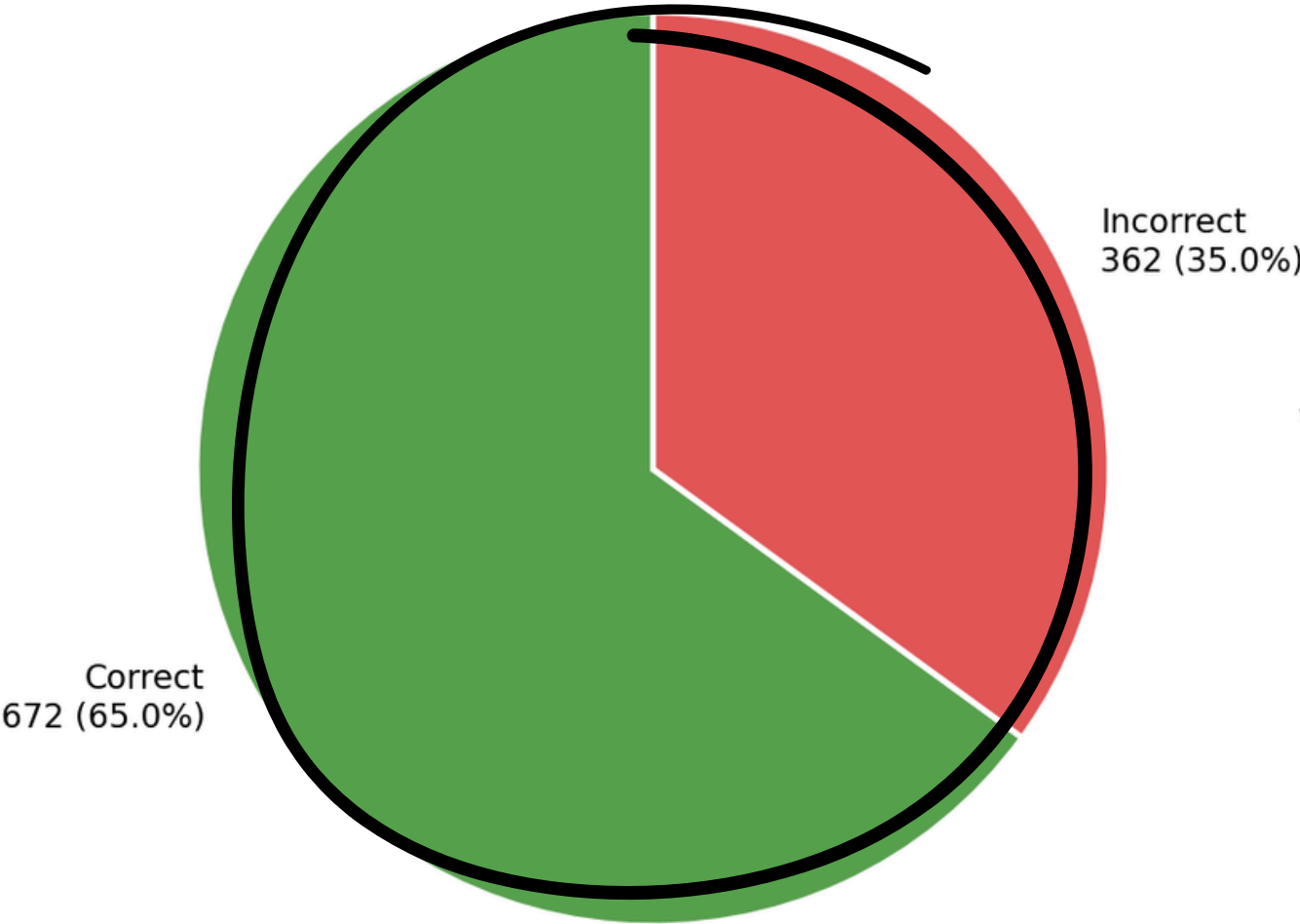
После всех проделанных шагов мы пришли к выводу, что все что могли выжать из модели в 1.5B параметров, мы выжали.

И дальше было принято решение проверить насколько лучше с данной задачей справляется модель побольше. Для нее мы тоже использовали более сложный промт, точно прописывая задачу и все связи между таблицами, делая ставку на то, что для более умной модели это даст прирост качества запросов.

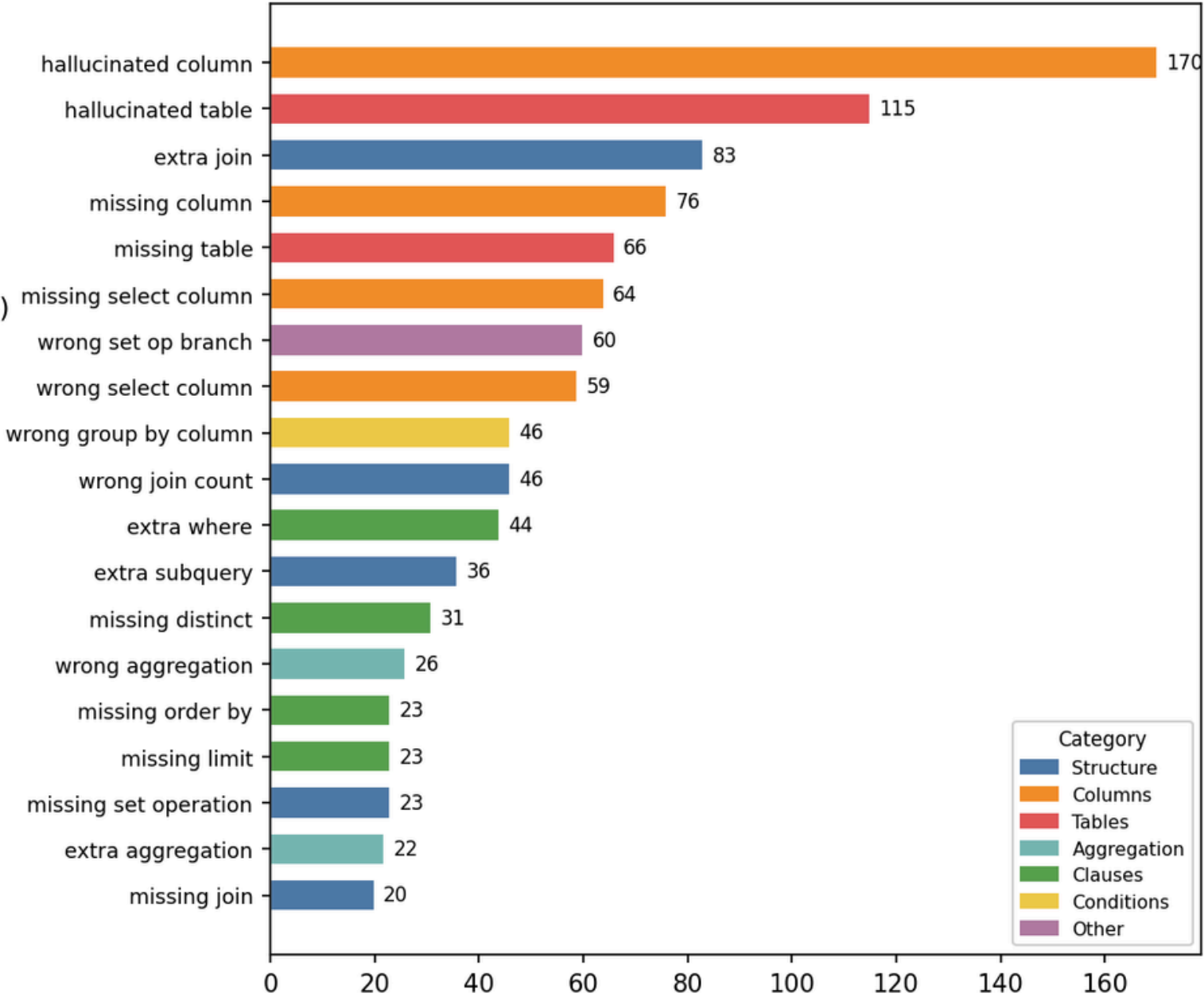
Новый улучшенный
инференс модели на 7B

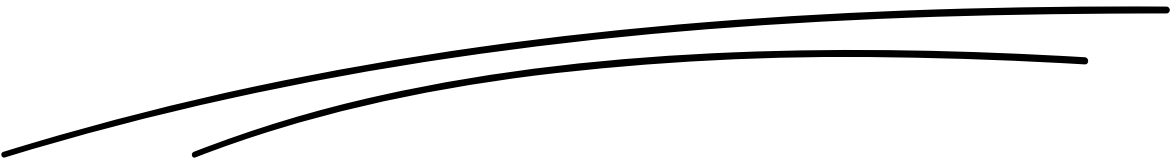
	easy	medium	hard	extra	all
count	248	446	174	166	1034
===== EXECUTION ACCURACY =====					
execution	0.891	0.836	0.730	0.627	0.798
===== EXACT MATCHING ACCURACY =====					
exact match	0.855	0.762	0.626	0.458	0.713
----- PARTIAL MATCHING ACCURACY -----					
select	0.980	0.933	0.958	0.912	0.945
select(no AGG)	0.988	0.938	0.958	0.918	0.950
where	0.823	0.828	0.733	0.559	0.758
where(no OP)	0.832	0.838	0.771	0.710	0.800
group(no Having)	0.941	0.869	0.857	0.845	0.865
group	0.941	0.820	0.833	0.831	0.833
order	0.955	0.982	0.938	0.878	0.930
and/or	1.000	0.998	0.994	0.944	0.989
IUEN	0.000	0.000	0.500	0.621	0.536
keywords	0.954	0.940	0.841	0.836	0.903

Overall Accuracy

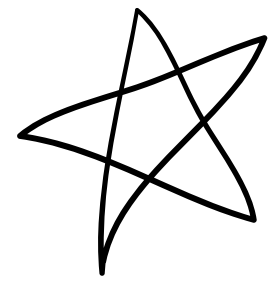


Error Type Distribution (Top 20)





Выводы



В рамках проекта мы исследовали применение QLoRA для адаптации моделей семейства Qwen2.5-Coder к задаче Text-to-SQL.

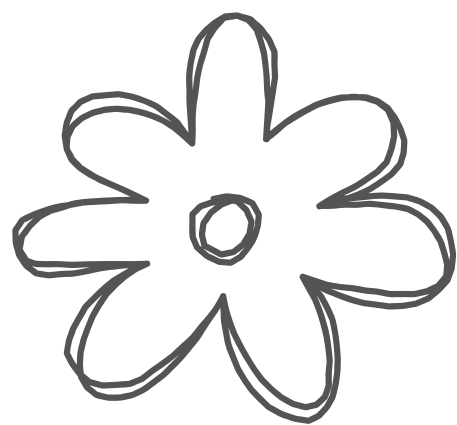
Сначала была дообучена компактная модель Qwen2.5-Coder-1.5B-Instruct. Базовая версия достигла **0.684** по Execution Accuracy, а улучшение inference-пайплайна позволило повысить результат примерно до **0.691**.

Дополнительные эксперименты показали, что усложнение обучения не всегда улучшает качество для маленькой модели. Более подробный schema-grounded prompt и weighted-аугментация сложных запросов снизили Execution Accuracy до 0.645. Это подтвердило необходимость перехода на другую модель.

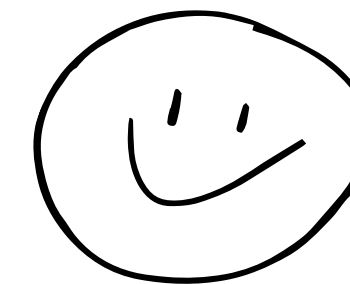
После этого тот же подход был и перенесён на Qwen2.5-Coder-7B-Instruct. Сравнение моделей позволило оценить, насколько увеличение числа параметров влияет на качество генерации SQL и устойчивость к более сложным запросам.

Отдельный анализ ошибок показал, что основные проблемы связаны с лишними JOIN, неверным выбором колонок, ошибками в условиях и сложных SQL-конструкциях. Причем модель 7B реже пропускает необходимые таблицы и колонки, а также реже генерирует несуществующие таблицы.

Для дополнительной проверки был реализован модуль SchemaGuard, который выявляет формальные ошибки и позволяет запускать повторную генерацию проблемных запросов.



Дальнейшие планы развития проекта



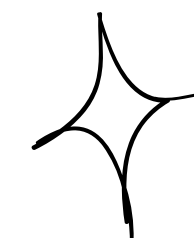
Следующим шагом может стать реализация ансамбля моделей. Отдельная модель-классификатор будет оценивать сложность запроса и выбирать подходящий маршрут обработки: простые запросы передавать более компактной и быстрой модели, а сложные - модели большего размера или специализированному модулю.

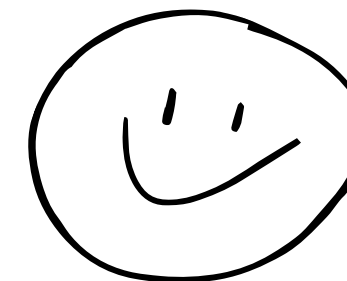
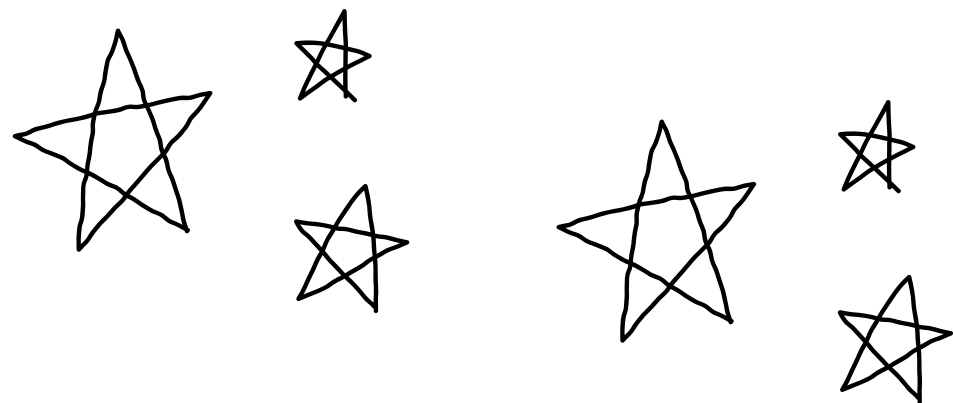
Также можно обучить отдельные модели для разных типов SQL-конструкций: запросов с JOIN, вложенных подзапросов, агрегатов и set-операций. Их результаты можно сравнивать и выбирать лучший вариант с помощью SchemaGuard и проверки исполнения SQL.

Ещё одно направление - обучение моделей большего размера, например 14B и выше, чтобы оценить, насколько дальнейшее увеличение числа параметров улучшает качество.

Дополнительно можно развивать repair-пайплайн: использовать SchemaGuard не только для диагностики, но и для автоматического выбора между несколькими сгенерированными кандидатами, а также для повторной генерации запросов с формальными ошибками.

Можно также попробовать расширить датасет действительно новыми примерами, а не клонированием





Спасибо за внимание!

